Identifying Road Anomalies Using YOLO

Ciara Sikking, Gene Zaleski, Nick Sokoya, Matt Gartland, and Eric Shelley

Department of Mathematics, Rowan University,
201 Mullica Hill Rd, Glassboro, NJ 08028, USA.

Abstract

We consider the problem of detecting road distresses from visual images captured obtained from a camera attached to a car bumper. This project aims to categorize images of roads into good or bad road images. Furthermore, if an anomaly is identified on a road surface, it will be labeled as a crack, pothole, marking, or patch. Our method is based on a Convolutional Neuron Network model named YOLO for image classification. Our method can categorize potholes, patches, and cracks quite well.

1 Statement of the industrial problem and summary of our results

{sec.1}

The overall goal of the project is to detect pavement distresses from visual images obtained from a low cost camera attached to a car bumper with an accuracy of at least 90 percent. This project was sponsored by TransDAC LLC: Intelligence Transportation Systems. TransDAC LLC is a start-up company in Glassboro, NJ. TransDAC LLC collaborates with Rowan College of Engineering at Rowan University in developing new transportation technologies. Dr. Parth Bharsar, the CTO of TransDAC LLC, was the industrial liaison for this project.

Potholes alone costs US drivers $3 billion annually in damages [1], therefore being able to detect these and other pavement distresses will be a very useful tool in improving the future of road transportation. To evaluate road conditions, TransDAC LLC is developing a low cost system which can be mounted in any vehicle and to capture images. Hence, there is a need to classify the captured images.

In this project we solve this classification problem using machine learning techniques. The program first deciphers whether an image is of a good road (no anomalies) or a bad road (at least one anomaly). If an image is classified as bad road, the existing anomalies will be segmented into one of five different classes: cracks, potholes, markings, patches, or "none of the above." The image classifier we use is You Only Look Once (YOLO), a

real-time object detection system which is based on the Convolutional Neuron Network (CNN) architecture.

To train YOLO, we first use the B-box label tool for labeling object bounding boxes in images. This tool is implemented with Python [5]. This allows us to manually identify anomalies in the training set via the bounding boxes. Each bounded box is labeled as one of the five aforementioned classes. These will be used to train our image classifier. Image classification is the process of taking an input (like a picture) and outputting a class (like "pothole") with the probability that the input belongs to a particular class.

Our results include the following: (i) We implemented YOLO on powerful GPU and hardware acceleration. (ii) We labeled and trained the algorithm with more than 2000 images. The trained algorithm was then tested against testing images. The testing accuracy is about 82% for potholes, 90% for patches, and 73% for cracks.

2 More details about the proposed method

{sec.2}

2.1 YOLO Classifier

{sec.2.1}

The image classifier used in this project is YOLO. To start, a user must input a training set of images. YOLO uses this data set to learn about the objects that it must detect. YOLO applies a single neural network to identify objects in the images. Neural networks transform a piece of information by processing it through a series of neuron layers. The neural network first transforms each image into a matrix. Within each neuron layer, a kernel matrix of weights is applied to the image matrix through matrix multiplication. The image matrix, when multiplied by the kernel matrix, is linearly transformed to a new matrix that is then sent to the next layer of neurons. This process continues until the matrix passes through the exit layer [7].

YOLO differs from other classifiers because after processing the whole image it divides it into small sections, or cells, for inspection. Each cell that contains an object produces a set of bounding boxes and class predictions to locate the object. Bounding boxes surround the objects in an image while class predictions give the predicted class for those objects. As mentioned, the inspection begins by breaking each image into a grid of small cells. Let $S \times S$ represent the grid dimensions that the image is broken into. If an object is found in a cell, then YOLO creates a set of bounding boxes and class predictions associated with that cell. Let B represent the set of bounding boxes and C represent the set of class predictions. Each cell can predict only one object regardless of how many bounding boxes it produces for that object. [4] An object is said to exist within a cell if its center falls within that cell. Each bounding box gets assigned a confidence score that reflects how confident YOLO is that an object exists in the associated bounding box. If no object exists in that cell, the confidence score will be zero. The equation for this score is as follows [7].

$$\text{bounding box confidence score} = Pr(Object) \times IOU_{pred}^{truth} \tag{2.1}$$

{eq.1}

Equation (2.1) defines the confidence score of an object existing in a specific bounding box. The $Pr(Object)$ term gives the probability that an object exists within a bounding box. This term will be either 0 if no object exist in the cell or 1 if an object exists in the cell. The IOU (Intersection over Union) term compares the coordinates of the YOLO-created bounding box (pred) and the coordinates of the bounding box that was manually created by a user (truth). The IOU function takes the area of intersection between the two bounding boxes and divides it by the area of union between them. Figure 1 provides a visual of this process [7].

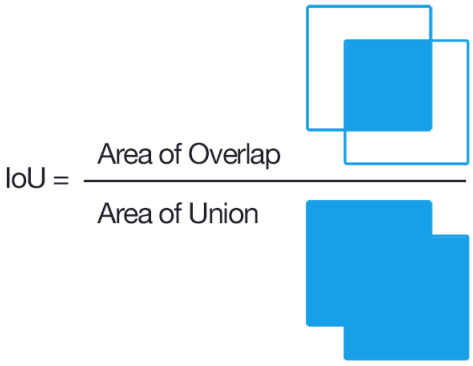


Figure 1: IOU equation {fig:IOU}

Each bounding box predicts 5 different components: $(x, y, w, h, confidence)$. The (x, y) components refer to the center of a bounded box surrounding a given object. The cell that the (x, y) coordinates exist in is associated with the bounding box and class predictions for the given object. The (w, h) components refer to the width and height of the bounded box; however, this value is relative to the entire image. All four of these components, (x, y, w, h), must be normalized to fall between 0 and 1 before the classifier can train [7].

Additionally, each cell produces a single set of class predictions. As the name suggests, class predictions aim to predict the class of the object within a specific cell. Class predictions give the probability that an object is in class i. In this case i may take the value of cracks, potholes, markings, patches, or none of the above. Each cell calculates a set of class predictions using the following calculation [7].

$$\text{conditional class probability} = Pr(Class_i|Object) \tag{2.2}$$ {eq.2}

Equation (2.2) gives the probability that an object is in class i given that an object exists. Since YOLO uses conditional probability, the classifier will not predict a class in an empty cell. This means that the class prediction provides a zero value for any cell that does not contain an object [7].

Each cell creates a set C class predictions. Since the image is broken into $S \times S$ cells,that makes $S \times S \times C$ class predictions in total. Adding the class predictions to the

bounding box predictions, we get a tensor output with the dimensions $S \times S \times (B*5+C)$ [4]. We multiply B by 5 because of the 5 components that each bounding box predicts [7]. Refer to Figure 2 for an example of how the tensor is formed, The picture below is broken into a 7×11 grid of cells ($S \times S$). Since the yellow cell contains the center of the pothole (red dot) it produces the bounding boxes and class predictions for the pothole. The yellow cell produced 2 blue bounding boxes around the pothole (B). In this example the cell will produce 5 class predictions (C) because of the 5 classes: cracks, potholes, markings, patches, and none of the above. YOLO's prediction tensor for the pothole would have a shape of $(S, S, B \times 5 + C) = (7, 11, 2 \times 5 + 5) = (7, 11, 15)$. In this tensor, 7 represents the amount of grid rows, 11 represents the amount of grid columns, and 15 represents the amount of predictions (the amount of bounding boxes and class predictions) for each cell. This tensor represents the amount of predictions that Figure 2 will have as whole.

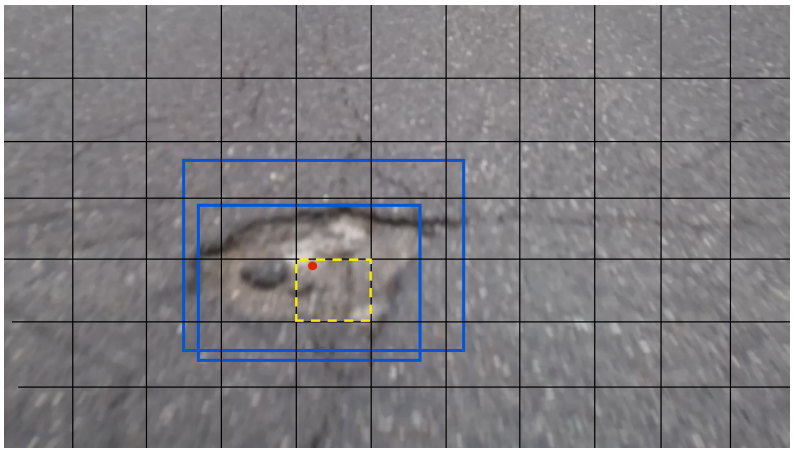


Figure 2: Example of bounded object {fig:Ex}

After YOLO processes a training data set, a user may feed the image classifier an image to test the training accuracy. If YOLO detects the object in the test image with high accuracy, the training was successful. YOLO will use equations (2.1) and (2.2) to gather information about the image and then multiply them together to predict the final bounding boxes for the objects in the image.

$$Pr(Class_i|Object) \times Pr(Object) \times IOU_{pred}^{truth} = Pr(Class_i) \times IOU_{pred}^{truth} \quad (2.3) \quad \{\texttt{eq.3}\}$$

Equation (2.3) provides the final class confidence score for each bounding box. The final bounding boxes will surround the object and label it with the class that YOLO believes the object is in [4].

Additionally YOLO applies the loss function. This function helps the image classifier track its learning process by provides a value that represents how close YOLO's predictions are to the truth [8]. See section 2.3 for more details about this function. YOLO's loss function uses the sum of the squares error method. This is a measure of the variance of

the measured data from the true mean of the data. YOLO utilizes this because the sum of the errors could be zero, on the average, since errors can be equally likely positive or negative. That would imply that there are no errors, which is not true. By squaring the errors this problem is overcome [3]. YOLO uses this method to compute the classification loss, the localization loss (errors between the predicted boundary box and the ground truth), and the confidence loss (the objectness of the box) [4].

Similarly to a regular CNN, YOLO's network structure has convolution, max pooling, and 2 fully connected layers at the end.

2.2 The Layers

{sec.2.2}

After the process described above, YOLO implements the convolution layers. Convolution layers are mostly used for 2-dimensional image data, which is perfectly suited to our needs. In general, a convolution is a linear operation that involves the multiplication of a set of weights with the input [2]. For 2D input, the multiplication is performed between an array of input data (the image converted into a matrix of numbers corresponding to the RGB color value) and a 2D array of weights, often called a filter or kernel. The purpose of a convolution layer is to extract features from the image. YOLO in particular implements the convolution layer 24 times (as 24 was found to be a good balance between speed and performance) in order to better define the desired features. In our case, our desired features are edges of cracks, potholes, etc. Therefore the more times the convolution process is performed, the more defined our edges will be, resulting in a higher IOU value.

The multiplication is applied between a filter-sized patch of the input and the filter. The filter is purposely made smaller than the input data. This is so the filter can be stepped column to column and row to row in the matrix and compare each value to its neighbor to find the biggest differences, i.e. where the edges are. This type of operation is called a "dot product." A breakdown of how a dot product works can be seen in Figure 3.

The dot product of two vectors $\mathbf{a} = [a_1, a_2, \ldots, a_n]$ and $\mathbf{b} = [b_1, b_2, \ldots, b_n]$ is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

Figure 3: Dot Product breakdown

{fig:Dot}

YOLO performs this multiplication between the filter-sized patch of the input and filter. As seen in Figure 3, after summation the process results in a single scalar value. These scalar values are combined in a matrix to form the convolution layer's output. The output of the convolution layer is called a feature map.

One limitation of a feature map output generated by a convolution layer is that they record the precise position of the features from the input. As a result, any small movements in the position of the features in the input image will result in a different feature map. For example cropping, rotating, or shifting an input image would all generate a different feature map. The best way to combat this issue is to add another layer to the network called a pooling layer. The pooling layer operates on each feature map separately. It will look at regions of the input image, detect the values for that region, and then reduce the region based on the type of pooling layer used. There are two main types of pooling layers, average and max. An average pooling method would calculate the average value of each region of the feature map. Max pooling on the other hand, determines the max value from the region of the feature map. Essentially a pooling layer is reducing the image to a summarized form. Not only does this help with the positioning of the features from the input, but it reduces the dimensionality of the network. Pooling layers enable the network to learn to recognize full objects in different shapes and positions. Figure 4 shows an overview of a convolution and pooling layer.

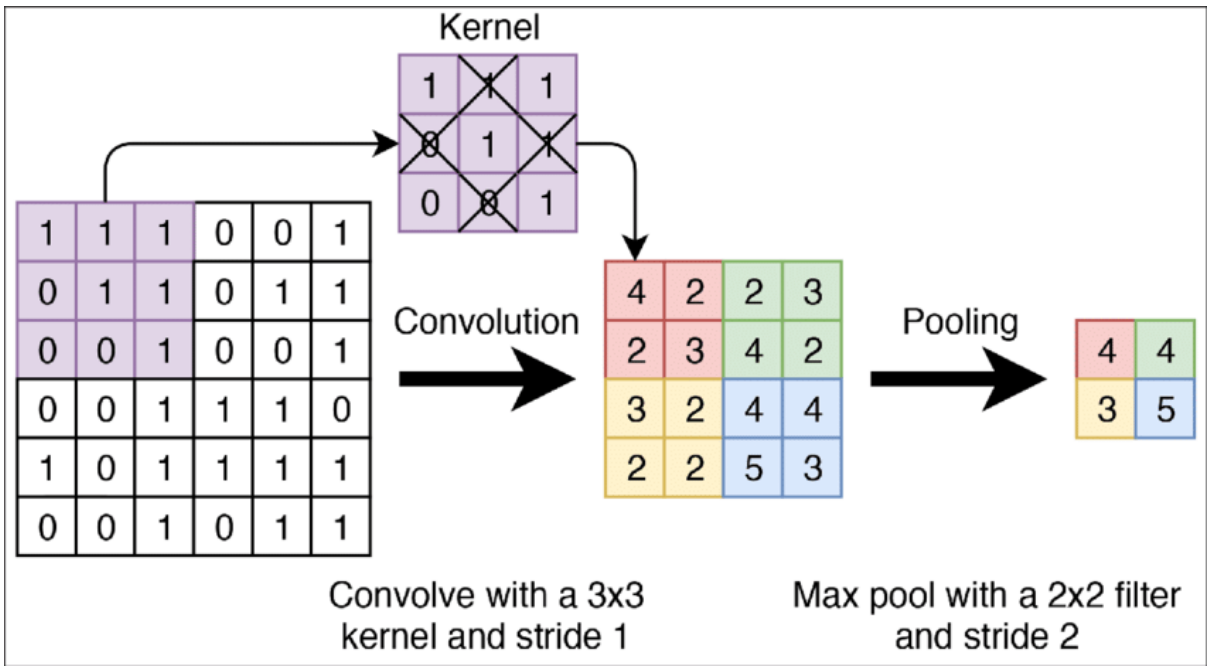


Figure 4: Convolution and Pooling Layers {fig:C&P}

Lastly, The Fully connected layer is the final layer in the CNN using convolution/pooling process to classify the image. It is broken up into three different stages; input layer(flatten), connected layer, and output layer. Fully connected input layer (flatten) takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage. The first fully connected layer takes the inputs from the feature analysis and applies weights to predict the correct label. Fully connected output layer

gives the final probabilities for each label. The output of convolution/pooling is flattened into a single vector of values, each representing a probability that a certain feature belongs to a label. Once the reader identifies characteristics that match a certain classifier it will then proceed to give the image a label.

To summarize the layers. We start with the convolution layer, after that is a max pooling layer. After these two layers are implemented multiple times (for more defined and accurate results) there are two fully connected layers at end. The full structure can be see in Figure 5. [4]

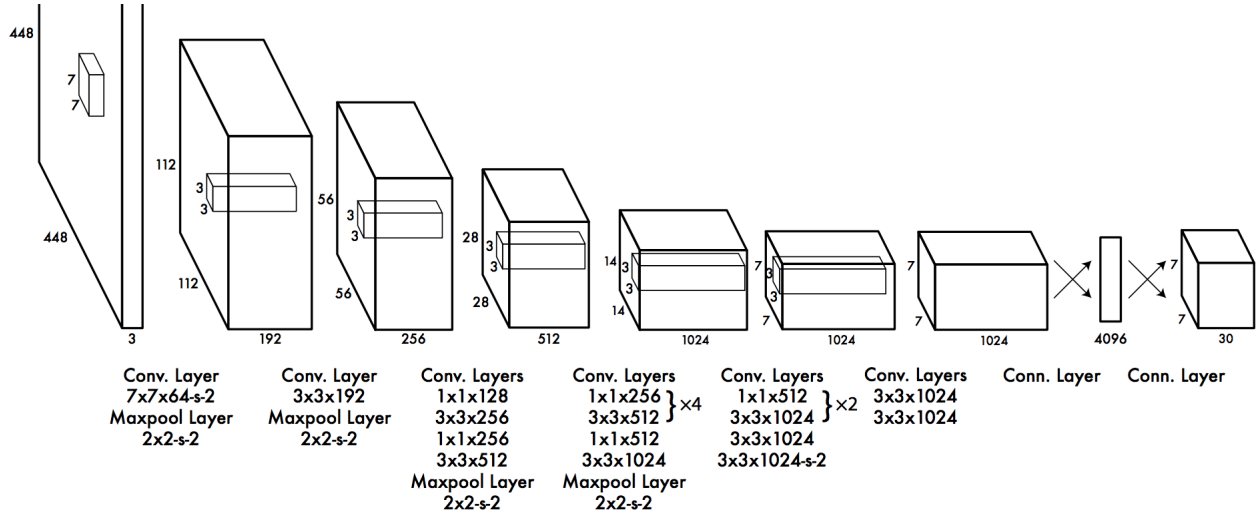


Figure 5: YOLO Structure {fig:Str}

2.3 The Loss Function {loss}

This function describes how close the results are to the ground truth. In other words a higher result of the loss function means that YOLO is further away from the correct predictions. On the other hand a lower result of the loss function means that YOLO is closer to the correct predictions, hence higher accuracy. The lost function is given by

$$\lambda coord \sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbb{1}_{ij}^{obj}(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + \lambda coord \sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbb{1}_{ij}^{obj}(\sqrt{w_i} - \sqrt{\hat{w}_i})^2$$
$$+ (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 + \sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbb{1}_{ij}^{obj}(C_i - \hat{C}_i)^2 + \lambda_n oobj \sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbb{1}_{ij}^{noobj}(C_i - \hat{C}_i)^2$$
$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{obj} \sum_{c \in classes} (p_i(c) - (\hat{p}_i(c))^2 \tag{2.4}$$ {eq.8}

The components of the loss function are explained as follows.

Part 1:

$$\lambda coord \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \tag{2.5}$$ {eq.4}

This term computes the loss related to the predicted bounding box position (x,y). The function computes a sum over each bounding box predictor ($j = 0..B$) of each grid cell ($i = 0..S^2$). $\mathbb{1}^{obj}$ is defined as follows: 1, if an object is present in grid cell i and the jth bounding box predictor is "responsible" for that prediction, otherwise, 0. Components (x, y) are the predicted bounding box position and $(\hat{x}, \hat{y})$ are the actual position from the training data [6].

Part 2:

$$\lambda coord \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \tag{2.6}$$ {eq.5}

This is the loss related to the predicted box width / height. It looks similar to the first one, except for the square root. This is because the error metric should reflect that small deviations in large boxes matter less than in small boxes. To partially address this we predict the square root of the bounding box width and height instead of the width and height directly [6].

Part 3:

$$\sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_n oobj \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 \tag{2.7}$$ {eq.6}

Here we compute the loss associated with the confidence score for each bounding box predictor. C is the confidence score and $\hat{C}$ is the IOU of the predicted bounding box with the ground truth. $\mathbb{1}^{obj}$ is equal to one when there is an object in the cell, and 0 otherwise. $\mathbb{1}^{noobj}$ is the opposite. The λ parameters that appear here and also in the first part are used to differently weight parts of the loss functions. This is necessary to increase model stability. The highest penalty is for coordinate predictions and the lowest for confidence predictions when no object is present [6].

Part 4:

$$\sum_{i=0}^{S^2} \mathbb{1}_{i}^{obj} \sum_{c \in classes} (p_i(c) - (\hat{p}_i(c))^2 \tag{2.8}$$ {eq.7}

It looks similar to a normal sum-squared error for classification, except for the $\mathbb{1}^{obj}$ term. This term is used so we don't penalize classification error when no object is present in the cell (hence the conditional class probability discussed earlier) [6].

2.4 Implementation

Training a YOLO image classifier on custom datasets is somewhat of an arduous process. Training and testing thousands of images to be classified is a big task for a computer,we created a Google Cloud Virtual Machine utilizing a NVIDIA Tesla P100 GPU. Following our machine setup, the first step of the training and testing process was bounding our images by hand for the purpose of training a YOLO classifier. We utilized the BBOX label tool, publicly available on github, to do this bounding.

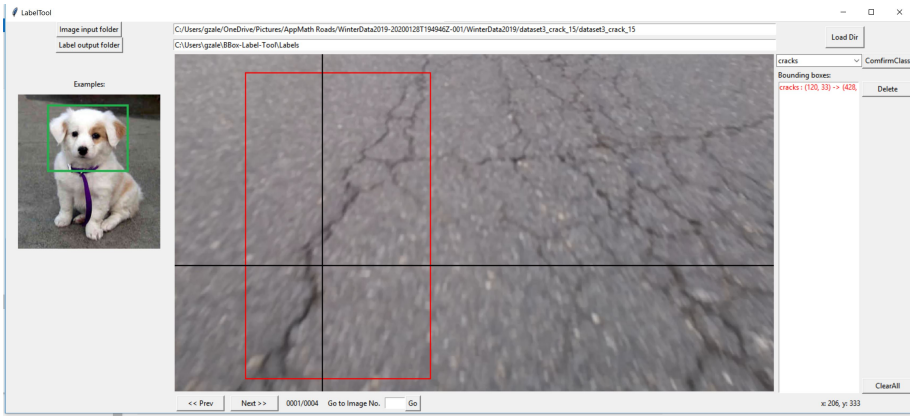


Figure 6: B-box label tool {fig:Bbox}

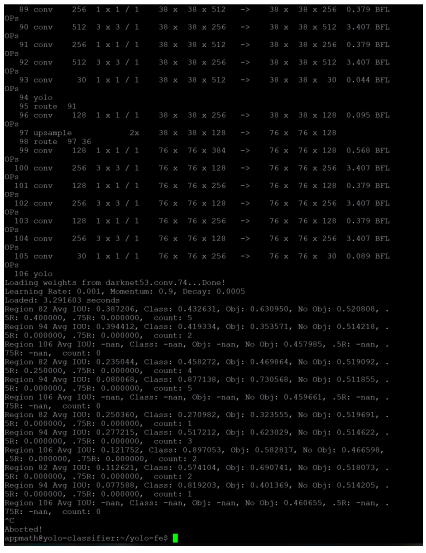


Figure 7: YOLO training data {fig:train}

We bounded 70 percent of our total data for training (2198 images) and reserved the rest (1079 images) for testing. The next step was training the YOLO classifier to

recognize each of these 5 classes: Potholes, Patches, Cracks, Markings, and None of the Above, which just represents a good section of road.

Training our image classifier was made easier by the use of our Ubuntu 16.04 Google Cloud Virtual Machine with a CUDA Accelerated NVIDIA Tesla P100 GPU. The NVIDIA Tesla is a top of the line GPU, and CUDA Acceleration allows for parallel processing within the GPU which increases processing capability. The yolo classifier we used can be found in different versions known as Darknet on GitHub. Through numerous testing, we found that the AlexeyAB branch of darknet tests result in proper labeling and different accuracy values due adjusted dependencies and bug fixes that aren't present in the Redmon darknet repo. However, the .weights classifier produced by the Redmon version has a higher accuracy value. To use our datasets in training, first we had to normalize our data to be between a value of (0,1), YOLO format and convert class names to numbers. This was done in python. Next, we sanitized our datasets to ensure there were matching numbers of images and bounding text files.

YOLO recommends 1000 iterations of testing for your image dataset, so for a dataset of 2000 images the training process took about 28 hours to train our data and produce a final ".weights" file for the purpose of image detection. Finally, once this process is complete we could run a detection run on our test dataset and produce results.

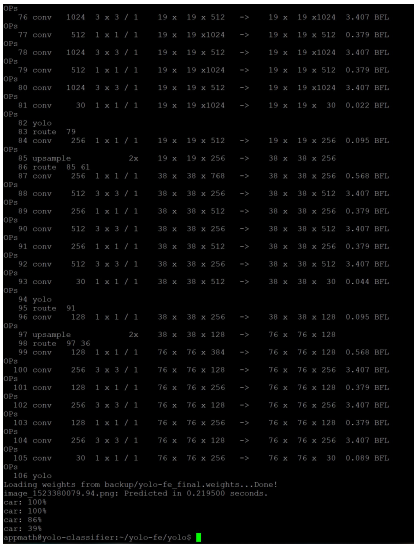


Figure 8: YOLO test data {fig:test}

3 More details about our results {sec.3}

From the above process, we could train our YOLO CNN classifier. When the training was complete, we could test and produce the following results (represented by this example):

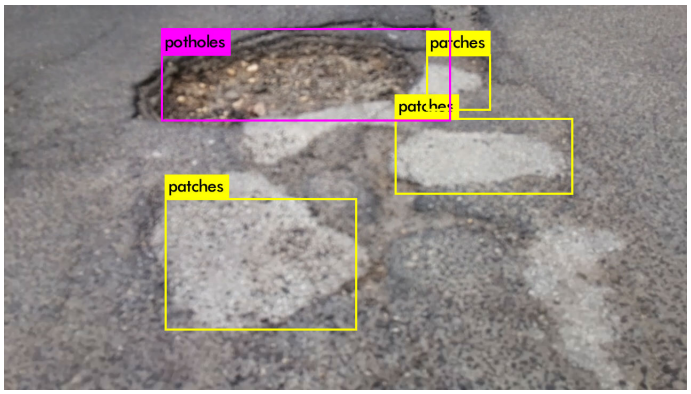


Figure 9: YOLO Result Image {fig:test}

Although training the classifier was aided by our powerful GPU and hardware acceleration, this was the most time consuming part. Testing an image with subjects took about .2 seconds.The accuracy metric we used as of this paper is Intersection over Union (IOU). The average IOU of our produced classifier is as shown in the result image 60.93 percent. However, there are more specific accuracy metrics we also produced. Mean Average Precision. Mean average precision is the mean of the average precision value for each class, which can be defined by the area under the curve created by plotting precision and recall values against each other.

Precision can be defined as:

$$(TruePositive)/((TruePositive) + (FalsePositive)) \tag{3.1}$$

Recall can be defined as:

$$(TruePositive)/((TruePositive) + (FalseNegative)) \tag{3.2}$$

True positive and negative are described by an IOU over a threshold, e.g. 0.5. As you can see, there are high values of average precision for certain classes and low values for others. Our Redmon Darknet classifier easily had the highest accuracy values, with no others breaking a 30 percent mean average precision or IOU. Our accuracy metrics are illustrated in Figure 10. As can be seen from Figure 10, our classifier achieved average accuracies of 81.98 percent for potholes, 90.51 percent for patches, 73.16 percent for cracks. The detection of markings is of only 1.18 percent accuracy. "None of the Above" was not detected in the testing. The average IoU is 60.93 percent.

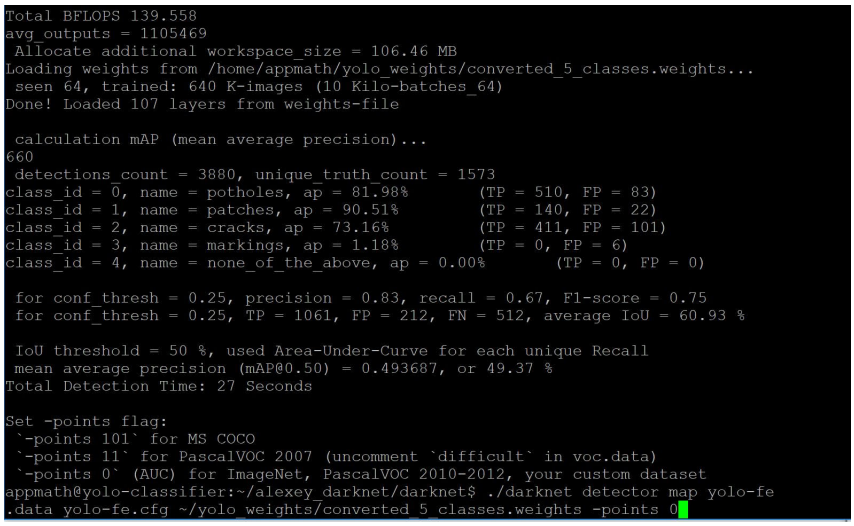


Figure 10: YOLO Accuracy {fig:test}

Potholes, Patches, and cracks all had fairly high accuracies, while Markings and None of the Above had low accuracies. Markings can be attributed to a combination of lower amount of training data on road markings and confusion with other anomalies. None of the Above classification can be considered the lack of classification as we did not bound none of the above as it was the entire picture of good road. Both of these classes factor into the IoU value that is produced by our image classifier, which explains the low average IoU obtained in our test.

4 Future Work {future}

Further extensions of this project include increasing the accuracy and precision of the image classifier by adding more pictures to the training set could work to ensure that the classifier is hitting its minimum of 90 percent accuracy. After this problem is overcome, the classifier may be expanded to classify road anomalies within pre-recorded videos or clips from television shows or movies.

5 Acknowledgments {sec.4}

PIC Math is a program of the Mathematical Association of America (MAA). Support for this MAA program is provided by the National Science Foundation (NSF grant DMS-1722275) and the National Security Agency (NSA).

We would like to thank Dr. Parth Bharsar, CTO of TransDAC LLC, for his support during this course. We also thank professor Thanh Nguyen for his mentoring during this project.

References

[1] AAA. Pothole damage cost us drivers $3 billion annually. 2020.

[2] J. Brownlee. How do convolutional layers work in deep learning neural networks? 1, June 2019.

[3] S. Ghobrial. What does the sum of squares error measure? March 2017.

[4] J. Hui. Real-time object detection with yolo, yolov2 and now yolov3. 1, March 2018.

[5] L. Janyst. bbox-label-tool. November 2017.

[6] M. Menegaz. Understanding YOLO. 1, March 2018.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. June 2016.

[8] P. Skalski. Deep dive into math behind deep networks. 1, August 2019.